10강

프로그래밍 언어론

복합타입

컴퓨터과학과 이관용 교수

학습목차



1 배열

2 문자열형

3 연관배열과 해싱

4 구조체와 공용체

5 포인터형과 참조형

01

배열

배열 array

- 동질형 데이터의 모음으로 구성된 타입
  - 원소 element → 배열의 각 데이터
  - 배열의 크기 → 배열이 가질 수 있는 원소의 개수

  ⇒ 사용자정의타입, 원시타입, 복합타입

- 각 원소는 첨자로 구별
  - 첨자 subscript → 첫 원소와의 상대적 위치

첨자	0	1	2	3
원소	1.0	5.7	-0.5	12.3

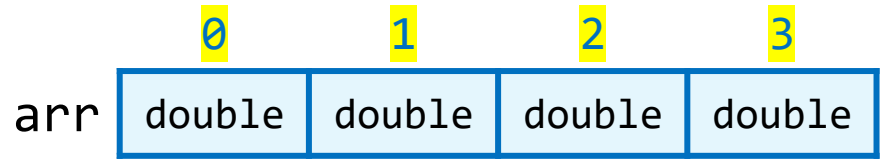
배열

데이터 집합

- 원소의 타입에 해당하는 데이터 집합이 배열의 크기만큼 모인 집합
- 원소의 타입과 배열의 크기에 따라 다양한 타입의 배열이 존재
  - 원소의 타입 → 단순타입 및 복합타입 모두 가능
  - 배열의 크기 → 양의 정수 범위

C, C++
```
double arr[4];
```

Java
```
double[] arr = new double[4];
```



배열

차원 dimension

- 배열에 사용되는 첨자의 개수
  - 1차원 배열 → [크기]
  - 2차원 배열 → [행] [열]
  - 3차원 배열 → [면] [행] [열]

    ⋮

  - 대부분의 PL → 차원의 제한이 없음

    단, Fortran 77 → 7차원, Fortran 2018 → 15차원

배열

차원

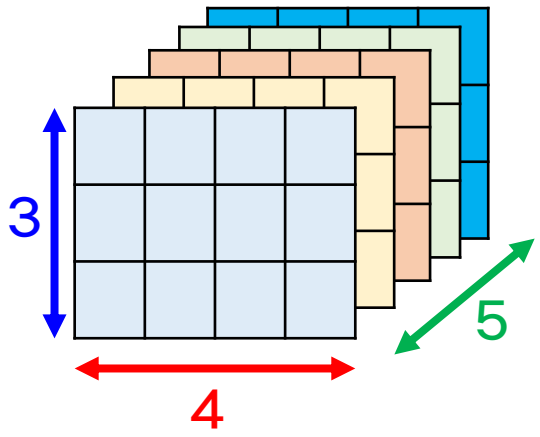
```
char a[4];
```

0	1	2	3
A	B	C	D

```
char b[3][4];
```

	0	1	2	3
0	A	B	C	D
1	E	F	G	H
2	I	J	K	L

```
char c[5][3][4];
```



0	0	1	2	3
0	A	B	C	D
1	E	F	G	H
2	I	J	K	L

`c[0][1][2]` → G

1	0	1	2	3
0	a	b	c	d
1	e	f	g	h
2	i	j	k	i

`c[1][2][3]` → i

배열의 구현 시 고려사항

(1) 저장공간의 확보

- 저장공간의 크기 = (원소 타입의 크기) × (배열의 크기)
- 정적 변수 → 초기 Fortran
- 스택 동적 변수
  - 배열의 크기는 정적 바인딩 또는 동적 바인딩으로 구분

```
int n=5;
char arr[n];
```

in C

```
char arr[5];
```

in C++

- 힙 동적 변수
  - 배열의 크기도 동적 바인딩

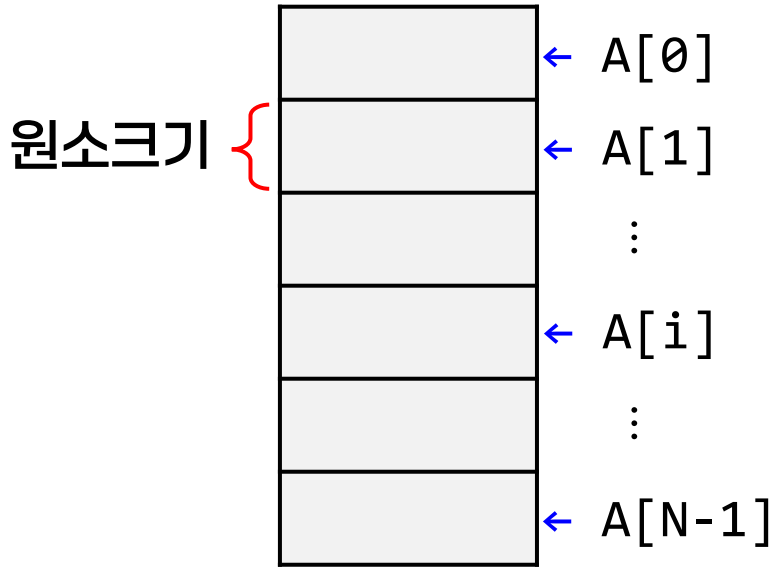
```
char[] arr = new char[5];
```

in Java

배열의 구현 시 고려사항

(2) 첨자를 통한 원소의 주소 계산

- 크기 N인 1차원 배열 A

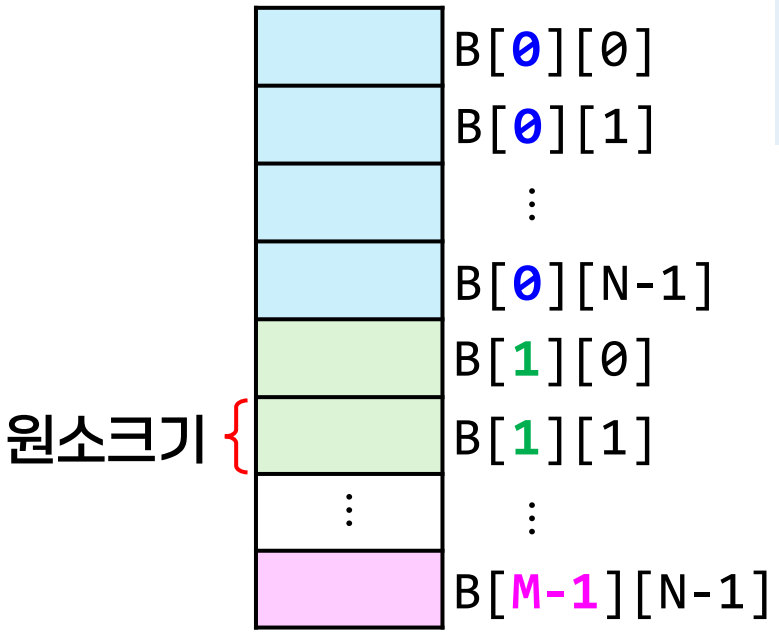


주소(A[i]) = 주소(A[0]) + i× 원소크기

배열의 구현 시 고려사항

(2) 첨자를 통한 원소의 주소 계산

- 크기 M×N인 2차원 배열 B
  - 행우선 저장 → 대부분의 PL



주소(B[i][j]) = 주소(B[0][0])
+ i×N×원소크기 + j×원소크기

```
char B[3][4];
```

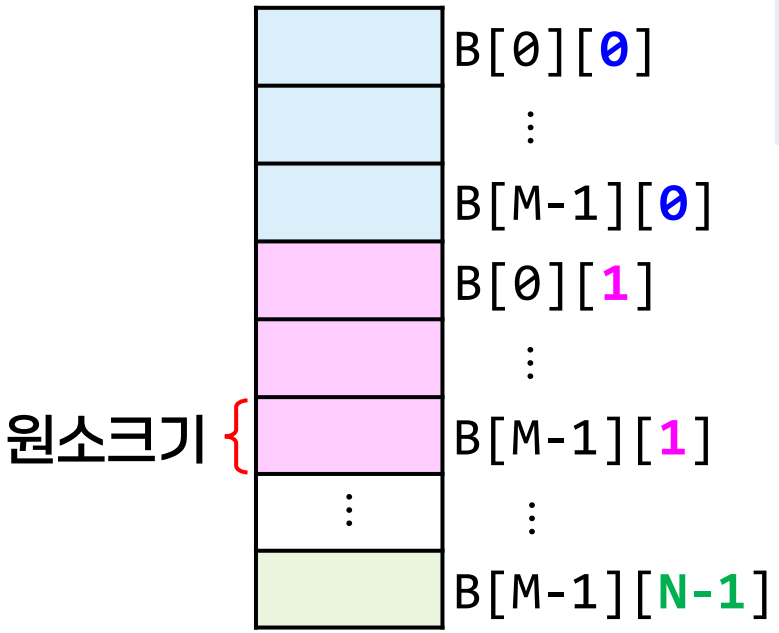
	0	1	2	3
0	a	b	c	d
1	e	f	g	h
2	i	j	k	l

주소(B[1][2])
= 주소(B[0][0])
+ 1×4×1
+ 2×1

배열의 구현 시 고려사항

(2) 첨자를 통한 원소의 주소 계산

- 크기 M×N인 2차원 배열 B
  - 열우선 저장 → Fortran



주소(B[i][j]) = 주소(B[0][0])
+ j×M×원소크기 + i×원소크기

```
char B[3][4];
```

	0	1	2	3
0	a	b	c	d
1	e	f	g	h
2	i	j	k	l

주소(B[1][2])
= 주소(B[0][0])
+ 2×3×1
+ 1×1

배열

연산 집합

- 배열 전체를 다루는 연산들이 모인 집합
- 개별 원소에 대한 연산 → 원소 타입의 연산 집합에 의존
- 배열 전체에 대한 연산 → PL에 따라 다름
  - C, C++, Java → 초기화 연산만 기본적으로 제공

```
double arr[4] = {1.1, 3.5, 17.6, -0.7};
```

  - Ada, Perl, Python → 대입 연산 등 제공(언어에 따라 비교 연산, 사칙 연산 지원)

02

문자열형

문자열형 string type

- 문자열 데이터를 다루는 데이터
  - 문자열 → 문자들의 나열
    - "KNOU", "good morning", "123", "a", …
- 데이터 집합
  - 모든 문자열이 모인 집합
    - C → 문자형 1차원 배열

```
char str[] = {'a', 'b', 'c', '\0'};
```

```
char str[] = "abc";
```

    - C++ → 문자형 1차원 배열 ('C 문자열'), string 클래스
    - Java → String 클래스 타입

```
String str = "abc";
```

문자열형

연산 집합

- PL마다 많은 차이가 존재
  - C, C++ → 기본적인 문자열 연산 없음
  - Java

    대입 연산 → `str2 = str;`

    연결 연산 → "Good" + "morning" ⇒ "Good morning"

03

연관배열과 해싱

연관배열 associative array

- 동질형 데이터의 순서 없는 모음으로 구성된 타입
  - 원소 간의 선후 관계, 위치 등이 의미 없음
- 각 원소는 키로 구별
  - 키 key → 각 원소의 절대 위치
    - 사용자가 직접 지정, 중복 불가
    - 원소는 중복 가능

(문자열형) 키	Tue	Mon	Wed	Sun
(정수형) 원소	2000	2000	2500	1000

연관배열

데이터 집합

- 키의 데이터 집합과 원소 타입의 데이터 집합의 쌍이 모인 집합
  - 키의 타입과 원소의 타입에 따라 다양한 타입의 연관배열 존재
- Perl → 해시 hash 타입

```
%walk = ('Tue' => 2000, 'Mon' => 2000,
         'Wed' => 2500, 'Sun' => 1000);
```

- C++ → map 타입

```
map<string, int> walk { {"Tue", 2000}, {"Mon", 2000},
                        {"Wed", 2500}, {"Sun ", 1000} };
```

연관배열

연산 집합

- 원소 참조, 원소 추가, 원소 삭제 등

	Perl	C++
원소 참조	`$walk{'Sun'}`	`walk["Sun"]`
원소 추가	`$walk{'Thu'} = 3000;`	`walk.insert({"Thu",3000});`
원소 삭제	`delete $walk{'Mon'};`	`walk.erase("Mon");`

04

구조체와 공용체

구조체 structure type

- "레코드형 record type"
- 다양한 타입의 데이터의 모음으로 구성된 타입
  - 동질형/이질형 모두 가능
  - 용어 '원소' 대신 **필드** field 사용 (C++에서는 멤버)
- 각 필드는 이름으로 구별
  - 필드 이름 → 각 필드의 타입으로 선언된 변수명

이름	name	birth	height	weight
필드	문자열	정수	실수	실수

구조체

데이터 집합

- 각 필드 타입에 해당되는 데이터 집합들이 모인 집합
  - 필드의 개수와 각 필드 타입에 따라 다양한 타입의 구조체 존재
  - 필드 타입 → 단순타입, 복합타입 모두 가능

C, C++ → struct 타입

```
struct person
{
  char name[20];
  int birth;
  double height;
  double weight;
};
```

Cobol → 레코드

```
01 person.
    05 name   PIC X(20).
    05 birth  PIC 9999.
    05 height PIC 999V9.
    05 weight PIC 999V9.
```

구조체

연산 집합

C 언어

```c
#include <stdio.h>
struct person
{
  char name[20];
  int birth;
  double height;
  double weight;
};

int main() {
  struct person m1={"ABC", 1980, 175.3, 69.8};
  struct person m2;
  m2 = m1;
  return 0;
}
```

구조체 정의

초기화

대입 연산

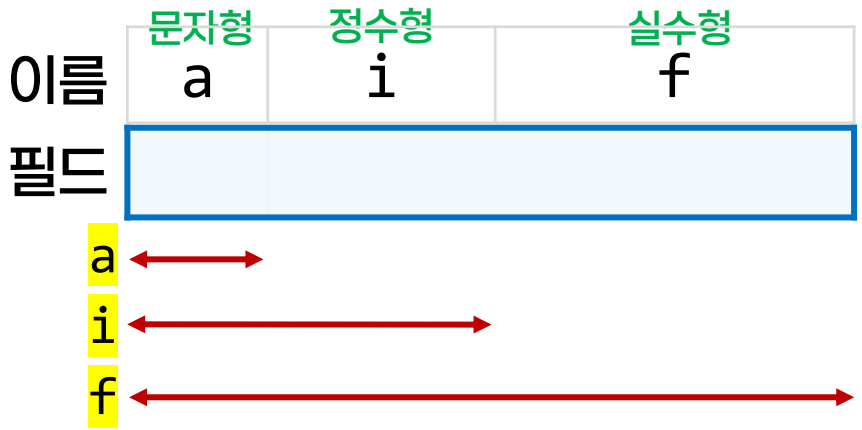
각 필드 접근 → 변수명.필드명

m2.birth

공용체 union type

- 저장공간을 공유하는 데이터의 모음으로 구성된 타입
- 정의 및 사용 방법은 구조체와 유사
- "저장공간 공유"

→ 한 필드의 값이 변경되면 다른 필드들의 값도 영향을 받음



공용체

데이터 집합

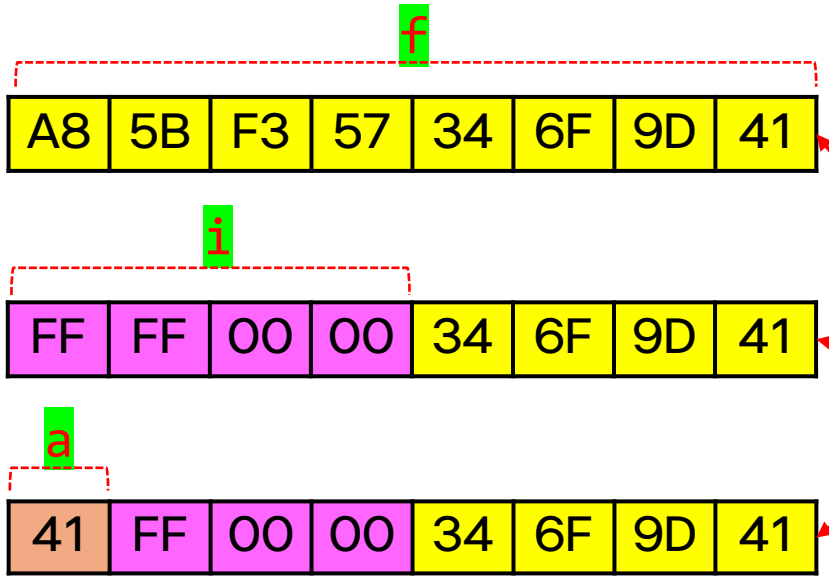
- 각 필드 타입에 해당하는 데이터 집합들의 합집합
  - 각 필드 타입의 데이터가 섞여서 하나의 집합을 구성
  - 각 필드 타입에 따라 다양한 타입의 공용체 존재
- C, C++ → union 타입

```
union uni
{
  int i;
  char a;
  double f;
};
```

공용체

연산 집합

- 개별 필드 타입의 연산 집합



C 언어

```c
#include <stdio.h>

union uni
{
  int i;
  char a;
  double f;
};

int main()
{
  union uni k;
  k.f = 123456789.987654321;
  k.i = 65535;
  k.a = 'A';
  printf ("%f\n", k.f);
  printf ("%d\n", k.i);
  return 0;
}
```

공용체 정의

```
123456768.000974
65345
```

공용체

공용 필드를 구별하는 방법

- 보통 태그 tag와 함께 사용 → 구조체의 하위 타입으로 공용체를 정의

```
struct Money {
 enum Kind { KRW, USD } tag;
 union Value {
  int won;
  float dollar;
 } amount;
};
```

```
struct Money {
 enum Kind { KRW, USD } tag;
 union {
  int won;
  float dollar;
 };
};
```

익명 공용체

```
struct Money m;
m.tag = KRW, m.amount.won = 1000;
m.tag = USD, m.amount.dollar = 1.50;
```

```
struct Money m;
m.tag = KRW, m.won = 1000;
m.tag = USD, m.dollar = 1.50;
```

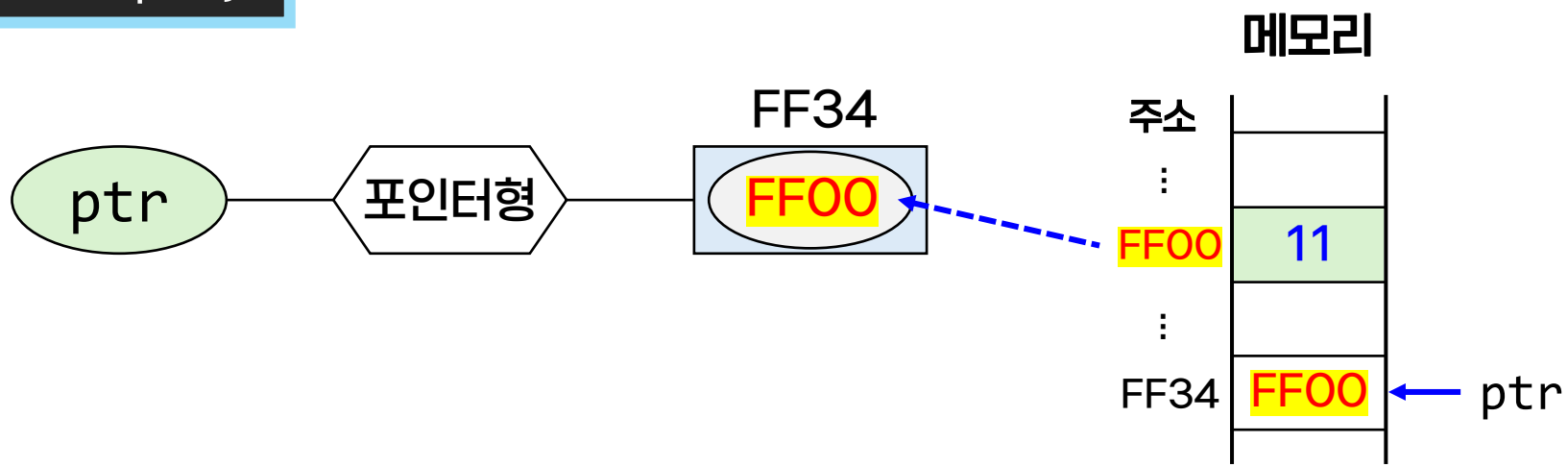
05

포인터형과 참조형

포인터형 pointer type

- 특정 데이터가 저장되는 주소 자체를 데이터로 다루는 타입
  - 특정 데이터의 타입은 사용자가 지정 → 단순타입, 복합타입 모두 가능

```
int *ptr;
```



포인터형

- 데이터 집합
  - 가용한 범위의 주소
    - 32비트 시스템 → 2^{32} 이내
    - 64비트 시스템 → 2^{64} 이내
  - C, C++

```
int *ptr;
```

→ "ptr은 int 타입의 데이터가 저장되는 메모리의 주소를 값으로 가짐"

포인터형

연산 집합

- 대입 연산, 덧셈/뺄셈 연산
- C, C++ → 간접참조 연산자 *

dereferencing operator

↓

사용 형식 → *변수명

"포인터 변수가 가리키는 주소에 저장된 데이터"

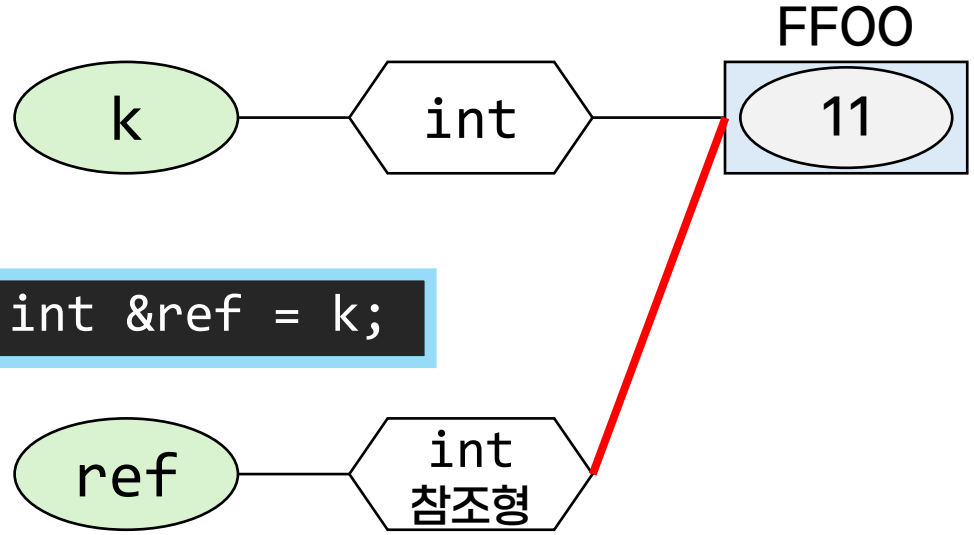
```
#include <stdio.h>

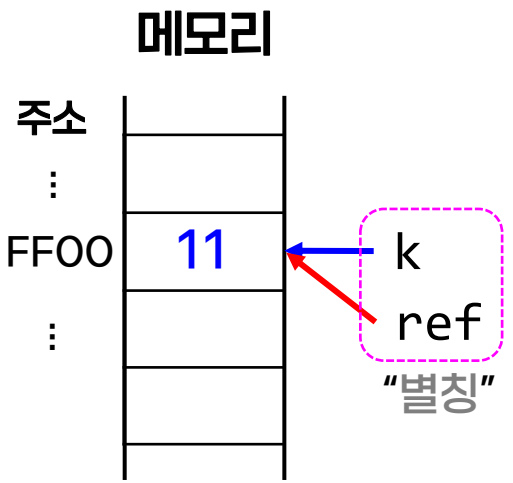
int main()
{
  int k[2] = {7, 5};
  int *ptr;    // 포인터 변수 선언
  ptr = k;     // 대입 연산
  printf ("%d\n", *ptr); 7
  ptr++;       // 덧셈 연산
  printf ("%d\n", *ptr); 5
  return 0;
}
```

참조형 reference type

- 사용자가 지정하는 별칭alias을 만드는 타입
  - 사용자가 지정한 타입의 데이터가 저장된 주소 자체를 자신의 주소로 사용
  - C, C++

```
int k = 11;
```





참조형

- 데이터 집합
  - 지정한 타입의 데이터 집합
- 연산 집합
  - 지정한 타입의 연산 집합

정리하기

배열

- 동질형 데이터의 모음, 사용자가 원소 타입과 크기 지정, 첨자로 원소 구분, 차원(첨자의 개수)
- 구현 고려사항 → 저장공간 확보, 첨자를 통한 원소의 주소 계산(2차원 이상은 행우선과 열우선 저장 방법)

문자열형

- 모든 문자열(문자들의 나열)이 모인 집합 + 연산 집합(PL마다 다름, C/C++은 기본 연산 없음)

연관배열과 해싱

- 동질형 데이터의 순서 없는 모음, 키로 원소 구분,
- 데이터 집합={(모든 키 타입 데이터, 원소 타입 데이터)}, 연산 집합(개별 원소에 대한 참조/추가/삭제 연산)
- Perl (해시 타입), C++(`map` 타입)

구조체와 공용체

- 구조체 → 동질형/이질형의 데이터 모음, 필드, 연산 집합(초기화, 대입 연산 등), C/C++(`struct` 타입)
- 공용체 → 저장공간을 공유하는 데이터 모음(한 필드 값 변경은 다른 필드 값에 영향), C/C++(`union` 타입)

포인터형과 참조형

- 포인터형 → 특정 데이터가 저장된 주소 자체를 데이터로 다루는 타입, C/C++(간접참조 연산자 *)
- 참조형 → 사용자가 지정한 변수의 별칭을 만들기 위한 타입, C++(변수 선언 시 참조 연산자 & 이용)

11강

다음시간안내 ▶▶▶

수식

-김진욱 교수